

SE LAB- 4

NAME : VINAY M RAMPUR

SRN : PES1UG24AM455

CLASS : 5TH AIML - H

LLM CHAT HISTORY

TASK -1

The screenshot shows a web browser window with multiple tabs. The active tab is ChatGPT, displaying a chat history on the left sidebar and a chat window on the right. The chat window contains a user message and a response from ChatGPT.

User Message:

I want to fix the first broken functionality in this Donkey Kong Python game. In the barrel movement logic, barrels currently have a 70% probability of taking a ladder. The required behavior is approximately 30%. Please inspect the relevant code, make the minimum necessary change to fix this probability, and explain briefly what you changed.

ChatGPT Response:

For Task 1, the relevant code is in `Barrel.update()` :

```
Python
if random.random() < 0.7:
```

The response includes a code snippet in a Python environment, showing the logic for the barrel movement. The code snippet is: `if random.random() < 0.7:`. The response also includes a "Run" button and a "Think" button.

The screenshot shows the ChatGPT web interface. On the left is a sidebar with navigation options: New chat, Scheduled, Library, Plugins, Codex, Explore, and Projects. The Projects section is expanded, showing 'SE PROJECT' with sub-items 'Phase 3 Tasks Estimate' and 'SAD Documentation Section', and 'STUDY MATE DBMS MINI PROJECT'. The main chat area displays a response from ChatGPT: 'This gives a 70% probability of taking the ladder. The required behavior is approximately 30%, so the minimum fix is:'. Below this is a code block for Python:

```
</> Python  
  
if random.random() < 0.3:
```

. The response continues with 'No other logic needs to change.' and 'What to do in VS Code', followed by 'Open game.py and change only:'. At the bottom of the chat area is a text input field with a '+' icon, a 'Think' button, and a microphone icon.

TASK - 2

The screenshot shows the ChatGPT web interface. The main chat area displays a response from ChatGPT: 'Implement the next feature from the repository README: theme_colour(score). Please inspect the existing code and README to determine the required behavior. Implement the function with the minimum necessary changes and keep it consistent with the existing project structure. Briefly explain what you changed and verify that it does not affect unrelated game logic.' Below this is a text input field with a '+' icon, a 'Think' button, and a microphone icon.

Use this implementation:

</> Python



```
def theme_color(score):  
    """Return an (r, g, b) background colour for the current score, or None for the default."""  
    if score <= 0:  
        return None  
  
    intensity = min(score / 1000, 1.0)  
    red = int(15 + 90 * intensity)  
    green = int(15 + 20 * intensity)  
    blue = 25  
  
    return (red, green, blue)
```

What this does

- `score == 0` → returns `None`, so the normal `Bg` is used.

+ Ask ChatGPT

Think



TASK - 3

Implement the next feature from the repository README:
`on_barrel_jumped(player, barrel)`.
Please inspect the existing code and README to determine the required behavior when the player successfully jumps over a barrel. Implement the feature with the minimum necessary changes and keep it consistent with the existing scoring and game logic. Briefly explain what you changed and verify that it does not affect unrelated game logic.



For Task 3, the README gives the required behavior clearly:

- `on_barrel_jumped(player, barrel)` is called **once when the player first jumps over a barrel**.
- The normal **100-point jump bonus** has already been awarded.
- The function exists purely for **side effects**; its `return` value is ignored.
- The README suggests a short-lived floating `"+100"` label or a sound. Since the project has no obvious

+ Ask ChatGPT

Think



Use this implementation:

</> Python



Run

```
def on_barrel_jumped(player, barrel):  
    """Called when the player clears a barrel; add a bonus effect here."""  
    floating_texts.append(  
        FloatingText("+100", barrel.x, barrel.y)  
    )
```

However, **don't paste that blindly**. The README doesn't specify that your project already has `FloatingText` or `floating_texts`. The correct implementation depends on the existing `game.py` structure.

Do this instead

Let your Vibe Coding AI inspect the existing code using exactly the prompt you wrote. It should determine how to implement the side effect using the classes/d ↓ structures already present.

The key requirement from the README is:

+ Ask ChatGPT

Think



TASK - 4

Implement the final feature from the repository README:

`score_multiplier(score)`.

Please inspect the existing scoring logic and README. Implement a simple score-based multiplier that rewards late-game play, while keeping the default multiplier at 1x for lower scores. Use the README's suggested behavior of returning 2x after an appropriate score threshold.

Make the minimum necessary changes, keep the implementation consistent with the existing project, and briefly explain the threshold and behavior you chose. Verify that it does not affect unrelated game logic.



Worked for 7s

Yes. I checked the **actual repository README** from your ↓ uploaded repo. For Task 4, it does **not specify an exact threshold**. It says to return 2x once the score passes some threshold to reward late-game play.

+ Ask ChatGPT

Think



So the implementation we discussed is reasonable:

</> Python

↗

📄

▶ Run

```
def score_multiplier(score):  
    """Return a multiplier applied to points earned from clearing a barrel, or None for the default 1x."""  
    if score >= 500:  
        return 2  
    return None
```

What this means

Because the existing code is:

</> Python

↗

📄

▶ Run

```
score += int(100 * (score_multiplier(score) or 1))
```

+

Ask ChatGPT

🧠 Think

🎤

🔊

the behavior becomes:

Score before jump	Multiplier	Points gained
0	1×	+100
100	1×	+100
400	1×	+100
500	2×	+200
600	2×	+200
1000	2×	+200

The 500 threshold is our design choice, since the RE ↓ ME leaves the threshold open.

